UNIT 2

RELATIONAL DATA MODELS

5.1 Relational Model Concepts

The relational model represents the database as a collection of relations. Informally, each relation resembles a table of values or, to some extent, a flat file of records. It is called a **flat file** because each record has a simple linear or flat structure.

When a relation is thought of as a table of values, each row in the table represents a collection of related data values. A row represents a fact that typically corresponds to a real-world entity or relationship. The table name and column names are used to help to interpret the meaning of the values in each row. For example, in table STUDENT each row represents facts about a particular student entity. The column names—Name, Student_Number, Class, and Major—specify how to interpret the data values in each row, based on the column each value is in. All values in a column are of the same data type.

In the formal relational model terminology, a row is called a tuple, a column header is called an attribute, and the table is called a relation.

The data type describing the types of values that can appear in each column is represented by a domain of possible values.

5.1.1 Domains, Attributes, Tuples, and Relations

- A **domain** D is a set of atomic values. By atomic we mean that each value in the domain is indivisible as far as the formal relational model is concerned.

Some examples of domains follow:

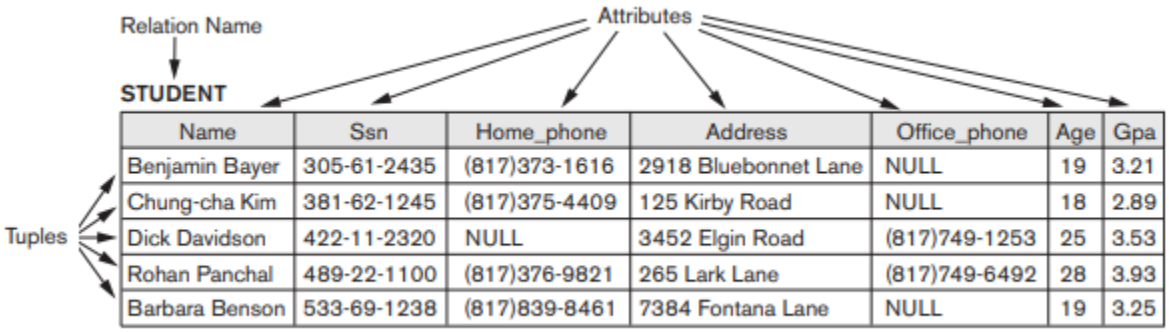
■ Usa_phone_numbers. The set of ten-digit phone numbers valid in the United States.

■ Names: The set of character strings that represent names of persons.

- The preceding are called logical definitions of domains. A **data type** or format is also specified for each domain. For example, the data type for Employee_ages is an integer number between 15 and 80. For Academic_department_names, the data type is the set of all character strings that represent valid department names.
- A **relation schema** R, denoted by R(A1, A2, … , An), is made up of a relation name R and a list of attributes, A1, A2, … , An. Each attribute Ai is the name of a role played by some domain D in the relation schema R. D is called the domain of Ai and is denoted by

dom(Ai). A relation schema is used to describe a relation; R is called the name of this relation. The degree (or arity) of a relation is the number of attributes n of its relation schema.
Example: A relation of degree seven, which stores information about university students, would contain seven attributes describing each student as follows:
STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)

A relation (or relation state) 4 r of the relation schema R(A1, A2, … , An), also denoted by r(R), is a set of n-tuples r = {t1, t2, … , tm}. Each n-tuple t is an ordered list of n values t =, where each value vi , 1 ≤ i ≤ n, is an element of dom (Ai) or is a special NULL value.
The ith value in tuple t, which corresponds to the attribute Ai , is referred to as t[Ai] or t.Ai (or t[i] if we use the positional notation). The terms relation intension for the schema R and relation extension for a relation state r(R) are also commonly used.



Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

Figure 5.1

5.1.2 Characteristics of Relations

- **Ordering of Tuples in a Relation.** A relation is defined as a set of tuples. Mathematically, elements of a set have no order among them; hence, tuples in a relation do not have any particular order. In other words, a relation is not sensitive to the ordering of tuples.
- **Ordering of Values within a Tuple and an Alternative Definition of a Relation**.
  According to the preceding definition of a relation, an n-tuple is an ordered list of n values, so the ordering of values in a tuple—and hence of attributes in a relation schema—is important. However, at a more abstract level, the order of attributes and their values is not that important as long as the correspondence between attributes and values is maintained.
  in a tuple unnecessary. In this definition, a relation schema R = {A1, A2, … , An} is a set of attributes (instead of an ordered list of attributes), and a relation state r(R) is a finite set of mappings r = {t1, t2, … , tm}, where each tuple ti is a mapping from R to D, and D is

the union (denoted by ∪) of the attribute domains; that is, D = dom(A1) ∪ dom(A2) ∪ … ∪ dom(An). In this definition, t[Ai] must be in dom(Ai) for 1 ≤ i ≤ n for each mapping t in r. Each mapping ti is called a tuple.

- **Values and NULLs in the Tuples.** Each value in a tuple is an atomic value; that is, it is not divisible into components within the framework of the basic relational model.
  An important concept is that of **NULL** values, which are used to represent the values of attributes that may be unknown or may not apply to a tuple.
- **Interpretation (Meaning) of a Relation.** The relation schema can be interpreted as a declaration or a type of assertion. For example, the schema of the STUDENT relation of Figure 5.1 asserts that, in general, a student entity has a Name, Ssn, Home_phone, Address, Office_phone, Age, and Gpa. Each tuple in the relation can then be interpreted as a fact or a particular instance of the assertion. For example, the first tuple in Figure 5.1 asserts the fact that there is a STUDENT whose Name is Benjamin Bayer, Ssn is 305-61-2435, Age is 19, and so on.
  An alternative interpretation of a relation schema is as a predicate; in this case, the values in each tuple are interpreted as values that satisfy the predicate. For example, the predicate STUDENT (Name, Ssn, …) is true for the five tuples in relation STUDENT of Figure 5.1.

5.1.3 Relational Model Notation

We will use the following notation in our presentation:

- A relation schema R of degree n is denoted by $R(A_1, A_2, \ldots, An)$.
- The uppercase letters Q, R, S denote relation names.
- The lowercase letters q, r, s denote relation states.
- The letters t, u, v denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation—the current relation state—whereas STUDENT(Name, Ssn, …) refers only to the relation schema.
- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation R.A—for example, STUDENT.Name or STUDENT.Age. This is because the same name may be used for two attributes in different relations. However, all attribute names in a particular relation must be distinct.
- An n-tuple t in a relation r(R) is denoted by t = , where vi is the value corresponding to attribute Ai. The following notation refers to component values of tuples: Both t[Ai] and t.Ai (and sometimes t[i]) refer to the value vi in t for attribute Ai. Both t[Au, Aw, … , Az] and t.(Au, Aw, … , Az), where Au, Aw, … , Az is a list of attributes from R, refer to the subtuple of values from t corresponding to the attributes specified in the list.

5.2 Relational Model Constraints and Relational Database Schemas

There are generally many restrictions or constraints on the actual values in a database state. These constraints are derived from the rules in the miniworld that the database represents.

Constraints on databases can generally be divided into three main categories:

1. Constraints that are inherent in the data model. We call these inherent model-based constraints or implicit constraints.

2. Constraints that can be directly expressed in the schemas of the data model, typically by specifying them in the DDL (data definition language). We call these schema-based constraints or explicit constraints.

3. Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs or in some other way. We call these application-based or semantic constraints or business rules.

5.2.1 Domain Constraints

Domain constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain dom(A). The data types associated with domains typically include standard numeric data types for integers (such as short integer, integer, and long integer) and real numbers (float and double-precision float). Characters, Booleans, fixed-length strings, and variable-length strings are also available, as are date, time, timestamp, and other special data types. Domains can also be described by a subrange of values from a data type or as an enumerated data type in which all possible values are explicitly listed.

5.2.2 Key Constraints and Constraints on NULL Values

Any such set of attributes SK is called a superkey of the relation schema R. A superkey SK specifies a uniqueness constraint that no two distinct tuples in any state r of R can have the same value for SK. Every relation has at least one default superkey— the set of all its attributes. A superkey can have redundant attributes, however, so a more useful concept is that of a key, which has no redundancy. A key k of a relation schema R is a superkey of R with the additional property that removing any attribute A from K leaves a set of attributes K' that is not a superkey of R any more. Hence, a key satisfies two properties:

1. Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key. This uniqueness property also applies to a superkey.

2. It is a minimal superkey—that is, a superkey from which we cannot remove any attributes and still have the uniqueness constraint hold. This minimality property is required for a key but is optional for a superkey.

In general, a relation schema may have more than one key. In this case, each of the keys is called a candidate key. For example, the CAR relation in Figure 5.4 has two candidate keys: License_number and Engine_serial_number. It is common to designate one of the candidate keys as the primary key of the relation. The other candidate keys are designated as unique keys

5.2.4 Entity Integrity, Referential Integrity, and Foreign Keys

The entity integrity constraint states that no primary key value can be NULL. This is because the primary key value is used to identify individual tuples in a relation.

Key constraints and entity integrity constraints are specified on individual relations. The referential integrity constraint is specified between two relations and is used to maintain the consistency among tuples in the two relations. Informally, the referential integrity constraint states that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation. For example, in Figure 5.6, the attribute Dno of EMPLOYEE gives the department number for which each employee works; hence, its value in every EMPLOYEE tuple must match the Dnumber value of some tuple in the DEPARTMENT relation.

To define referential integrity more formally, first we define the concept of a foreign key. The conditions for a foreign key, given below, specify a referential integrity constraint between the two relation schemas R1 and R2. A set of attributes FK in relation schema R1 is a foreign key of R1 that references relation R2 if it satisfies the following rules:

1. The attributes in FK have the same domain(s) as the primary key attributes PK of R2; the attributes FK are said to reference or refer to the relation R2.

2. A value of FK in a tuple t1 of the current state r1(R1) either occurs as a value of PK for some tuple t2 in the current state r2(R2) or is NULL. In the former case, we have t1[FK] = t2[PK], and we say that the tuple t1 references or refers to the tuple t2.

In this definition, R1 is called the referencing relation and R2 is the referenced relation. If these two conditions hold, a referential integrity constraint from R1 to R2 is said to hold.

5.2.5 Other Types of Constraints

The preceding integrity constraints are included in the data definition language because they occur in most database applications. Another class of general constraints, sometimes called semantic integrity constraints, are not part of the DDL and have to be specified and enforced in

a different way. Examples of such constraints are the salary of an employee should not exceed the salary of the employee's supervisor and the maximum number of hours an employee can work on all projects per week is 56. Such constraints can be specified and enforced within the application programs that update the database, or by using a general-purpose constraint specification language

5.3 Update Operations, Transactions, and Dealing with Constraint Violations

In this section, we concentrate on the database modification or update operations. There are three basic operations that can change the states of relations in the database: Insert, Delete, and Update (or Modify). **Insert** is used to insert one or more new tuples in a relation, **Delete** is used to delete tuples, and **Update** (or Modify**)** is used to change the values of some attributes in existing tuples. Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated. In this section we discuss the types of constraints that may be violated by each of these operations and the types of actions that may be taken if an operation causes a violation.

5.3.1 The Insert Operation

Insert can violate any of the four types of constraints.

1. Domain constraints can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type
   Example:
   Error: This insertion violates the entity integrity constraint (NULL for the primary key Ssn), so it is rejected.
2. Key constraints can be violated if a key value in the new tuple t already exists in another tuple in the relation r(R).
   Example:
   Error:This insertion violates the key constraint because another tuple with the same Ssn value already exists in the EMPLOYEE relation, and so it is rejected.
3. Entity integrity can be violated if any part of the primary key of the new tuple t is NULL.
   Example:
   Error: This insertion violates the referential integrity constraint specified on Dno in EMPLOYEE because no corresponding referenced tuple exists in DEPARTMENT with Dnumber = 7.
4. Referential integrity can be violated if the value of any foreign key in t refers to a tuple that does not exist in the referenced relation.

Example:
Error: This insertion satisfies all constraints, so it is acceptable.

In the first operation, the DBMS could ask the user to provide a value for Ssn, and could then accept the insertion if a valid Ssn value is provided. In operation 3, the DBMS could either ask the user to change the value of Dno to some valid value (or set it to NULL), or it could ask the user to insert a DEPARTMENT tuple with Dnumber = 7 and could accept the original insertion only after such an operation was accepted. Notice that in the latter case the insertion violation can cascade back to the EMPLOYEE relation if the user attempts to insert a tuple for department 7 with a value for Mgr_ssn that does not exist in the EMPLOYEE relation.

5.3.2 The Delete Operation

The Delete operation can violate only referential integrity. This occurs if the tuple being deleted is referenced by foreign keys from other tuples in the database. To specify deletion, a condition on the attributes of the relation selects the tuple (or tuples) to be deleted. Here are some examples.

■ Operation: Delete the WORKS_ON tuple with Essn = '999887777' and Pno = 10.

Result: This deletion is acceptable and deletes exactly one tuple.

■ Operation: Delete the EMPLOYEE tuple with Ssn = '999887777'.

Result: This deletion is not acceptable, because there are tuples in WORKS_ON that refer to this tuple. Hence, if the tuple in EMPLOYEE is deleted, referential integrity violations will result.

■ Operation: Delete the EMPLOYEE tuple with Ssn = '333445555'. Result: This deletion will result in even worse referential integrity violations, because the tuple involved is referenced by tuples from the EMPLOYEE, DEPARTMENT, WORKS_ON, and DEPENDENT relations.

Several options are available if a deletion operation causes a violation. The first option, called restrict, is to reject the deletion. The second option, called cascade, is to attempt to cascade (or propagate) the deletion by deleting tuples that reference the tuple that is being deleted. For example, in operation 2, the DBMS could automatically delete the offending tuples from WORKS_ON with Essn = '999887777'. A third option, called set null or set default, is to modify the referencing attribute values that cause the violation; each such value is either set to NULL or changed to reference another default valid tuple

5.3.3 The Update Operation

The Update (or Modify) operation is used to change the values of one or more attributes in a tuple (or tuples) of some relation R. It is necessary to specify a condition on the attributes of the relation to select the tuple (or tuples) to be modified. Here are some examples.

■ Operation: Update the salary of the EMPLOYEE tuple with Ssn = '999887777' to 28000.
Result: Acceptable.

■ Operation: Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 1.

Result: Acceptable.

■ Operation: Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 7.

Result: Unacceptable, because it violates referential integrity.

■ Operation: Update the Ssn of the EMPLOYEE tuple with Ssn = '999887777' to '987654321'.
Result: Unacceptable, because it violates primary key constraint by repeating a value that already exists as a primary key in another tuple; it violates referential integrity constraints because there are other relations that refer to the existing value of Ssn.

If a foreign key attribute is modified, the DBMS must make sure that the new value refers to an existing tuple in the referenced relation (or is set to NULL). Similar options exist to deal with referential integrity violations caused by Update as those options discussed for the Delete operation.

5.3.4 The Transaction Concept

A database application program running against a relational database typically executes one or more transactions. A transaction is an executing program that includes some database operations, such as reading from the database, or applying insertions, deletions, or updates to the database. At the end of the transaction, it must leave the database in a valid or consistent state that satisfies all the constraints specified on the database schema.

For example, a transaction to apply a bank withdrawal will typically read the user account record, check if there is a sufficient balance, and then update the record by the withdrawal amount.

A large number of commercial applications running against relational databases in online transaction processing (OLTP) systems are executing transactions at rates that reach several hundred per second.

14. BASICS OF FUNCTIONAL DEPENDENCIES AND NORMALIZATION FOR RELATIONAL DATABASES

14.1 Redundant Information in Tuples and Update Anomalies

A database anomaly is a fault in a database that usually emerges as a result of shoddy planning and storing everything in a flat database. In most cases, this is removed through the normalization procedure, which involves the joining and splitting of tables. The purpose of the normalization process is to minimize the negative impacts of generating tables that would generate anomalies in the DB.

Example

Consider a manufacturing firm that keeps worker information in a table called employee, which has four columns: w_id for the employee's id, w_name for the employee's name, w_address for the employee's address, and w_dept for the employee's department. The table will look like this at some point:

w_id	w_name	w_address	w_dept
201	David	Delhi	F001
201	David	Delhi	F002
223	Mike	Agra	F890
266	Berry	Chennai	F900
266	Berry	Chennai	F004

Type of Anomalies in DBMS

1. Update

2. Insert

3. Delete

Anomalies in databases can be, thus, divided into three major categories:

Update Anomaly

Employee David has two rows in the table given above since he works in two different departments. If we want to change David's address, we must do so in two rows, else the data would become inconsistent.

If the proper address is updated in one of the departments but not in another, David will have two different addresses in the database, which is incorrect and leads to inconsistent data.

Insert Anomaly

If a new worker joins the firm and is currently unassigned to any department, we will be unable to put the data into the table because the w_dept field does not allow nulls.

Delete Anomaly

If the corporation closes the department F890 at some point in the future, deleting the rows with w_dept as F890 will also erase the information of employee Mike, who is solely assigned to this department.

14.2 Functional Dependencies

The single most important concept in relational schema design theory is that of a functional dependency. In this section we formally define the concept

14.2.1 Definition of Functional Dependency

Definition. A functional dependency, denoted by X → Y, between two sets of attributes X and Y that are subsets of R specifies a constraint on the possible tuples that can form a relation state r of R. The constraint is that, for any two tuples t1 and t2 in r that have t1[X] = t2[X], they must also have t1[Y] = t2[Y].

For example:

Assume we have an employee table with attributes: Emp_Id, Emp_Name, Emp_Address.

Here Emp_Id attribute can uniquely identify the Emp_Name attribute of employee table because if we know the Emp_Id, we can tell that employee name associated with it.

Functional dependency can be written as:

1. Emp_Id → Emp_Name

We can say that Emp_Name is functionally dependent on Emp_Id.

Note the following:

■ If a constraint on R states that there cannot be more than one tuple with a given X-value in any relation instance r(R)—that is, X is a candidate key of R—this implies that X → Y for any subset of attributes Y of R (because the key constraint implies that no two tuples in any legal state r(R) will have the same value of X). If X is a candidate key of R, then X → R.

■ If X → Y in R, this does not say whether or not Y → X in R.

14.3 Normal Forms Based on Primary Keys

Having introduced functional dependencies, we are now ready to use them to specify how to use them to develop a formal methodology for testing and improving relation schemas.

14.3.1 Normalization of Relations

The normalization process, as first proposed by Codd (1972a), takes a relation schema through a series of tests to *certify* whether it satisfies a certain **normal form**.

Initially, Codd proposed three normal forms, which he called first, second, and third normal form. A stronger definition of 3NF—called Boyce-Codd normal form (BCNF)—was proposed later by Boyce and Codd.

Normalization of data can be considered a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of (1) minimizing redundancy and (2) minimizing the insertion, deletion, and update anomalies .It can be considered as a "filtering" or "purification" process to make the design have successively better quality. An unsatisfactory relation schema that does not meet the condition for a normal form—the normal form test—is decomposed into smaller relation schemas that contain a subset of the attributes and meet the test that was otherwise not met by the original relation. Thus, the normalization procedure provides database designers with the following:

■ A formal framework for analyzing relation schemas based on their keys and on the functional dependencies among their attributes

■ A series of normal form tests that can be carried out on individual relation schemas so that the relational database can be normalized to any desired degree

Definition. The **normal form** of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

14.3.2 Practical Use of Normal Forms

Most practical design projects in commercial and governmental environment acquire existing designs of databases from previous designs, from designs in legacy models, or from existing files. They are certainly interested in assuring that the designs are good quality and sustainable over long periods of time. Existing designs are evaluated by applying the tests for normal forms, and normalization is carried out in practice so that the resulting designs are of high quality and meet the desirable properties stated previously

Definition. **Denormalization** is the process of storing the join of higher normal form relations as a base relation, which is in a lower normal form.

14.3.3 Definitions of Keys and Attributes Participating in Keys

Definition. A **superkey** of a relation schema $R = \{A_1, A_2, \ldots, A_n\}$ is a set of attributes $S \subseteq R$ with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]. A **key** K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey anymore.

If a relation schema has more than one key, each is called a **candidate key**. One of the candidate keys is arbitrarily designated to be the **primary key,** and the others are called secondary keys

Definition. An attribute of relation schema R is called a **prime attribute** of R if it is a member of some candidate key of R. An attribute is called nonprime if it is not a prime attribute—that is, if it is not a member of any candidate key.

14.3.4 First Normal Form

It states that the domain of an attribute must include only atomic (simple, indivisible) values and that the value of any attribute in a tuple must be a single value from the domain of that attribute.

A table is in 1 NF if:

1. There are only Single Valued Attributes.
2. Attribute Domain does not change.
3. There is a unique name for every Attribute/Column.
4. The order in which data is stored does not matter.

Example:

Serial_No.	Titles	Courses
11	Xkon	CN, OS
12	Ykon	Java
13	Zkon	C++, C

In the above table, Course is a multi-valued attribute so it is not in 1NF.

Below Table is in 1NF as there is no multi-valued attribute:

Serial_No.	Titles	Courses
11	Xkon	CN
11	Xkon	OS
12	Ykon	Java
13	Zkon	C++
13	Zkon	C

14.3.5 Second Normal Form

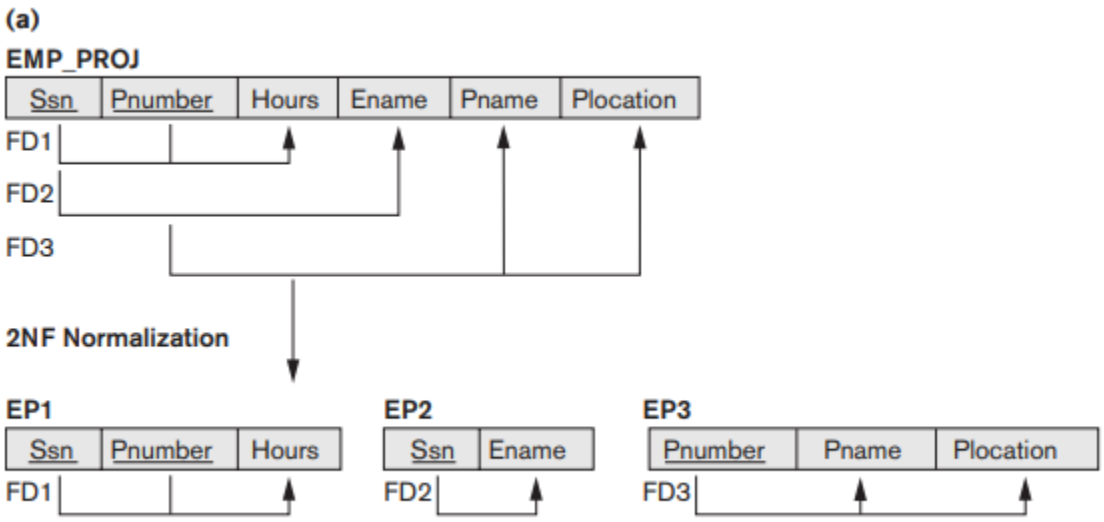
Second Normal Form (2NF) is based on the concept of full functional dependency. Second Normal Form applies to relations with composite keys, that is, relations with a primary key composed of two or more attributes.

To be in second normal form,

- a relation must be in first normal form
- relation must not contain any partial dependency.

A relation is in 2NF if it has No Partial Dependency, i.e., no non-prime attribute (attributes which are not part of any candidate key) is dependent on any proper subset of any candidate key of the table.

Example 1:



14.3.6 Third Normal Form

Definition. According to Codd's original definition, a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

Note – If A->B and B->C are two FDs then A->C is called transitive dependency.

The normalization of 2NF relations to 3NF involves the removal of transitive dependencies. If a transitive dependency exists, we remove the transitively dependent attribute(s) from the relation by placing the attribute(s) in a new relation along with a copy of the determinant.

Consider the examples given below.

Example-1:
In relation STUDENT given in Table 4,

STUD_NO	STUD_NAME	STUD_STATE	STUD_COUNTRY	STUD_AGE
1	RAM	HARYANA	INDIA	20
2	RAM	PUNJAB	INDIA	19
3	SURESH	PUNJAB	INDIA	21

Table 4

FD set:
{STUD_NO -> STUD_NAME, STUD_NO -> STUD_STATE, STUD_STATE -> STUD_COUNTRY, STUD_NO -> STUD_AGE}

Candidate Key:
{STUD_NO}

For this relation in table 4, STUD_NO -> STUD_STATE and STUD_STATE -> STUD_COUNTRY are true. So STUD_COUNTRY is transitively dependent on STUD_NO. It violates the third normal form. To convert it in third normal form, we will decompose the relation STUDENT (STUD_NO, STUD_NAME, STUD_PHONE, STUD_STATE, STUD_COUNTRY_STUD_AGE) as:

STUDENT (STUD_NO, STUD_NAME, STUD_PHONE, STUD_STATE, STUD_AGE)

STATE_COUNTRY (STATE, COUNTRY)

Example-2:
Consider relation R(A, B, C, D, E)

```
A -> BC,

CD -> E,

B -> D,

E -> A
```

All possible candidate keys in above relation are {A, E, CD, BC} All attribute are on right sides of all functional dependencies are prime.

14.5 Boyce-Codd Normal Form

BCNF (Boyce Codd Normal Form) is the advanced version of 3NF. A table is in BCNF if every functional dependency X->Y, X is the super key of the table. For BCNF, the table should be in 3NF, and for every FD. LHS is super key.

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example: In this example, we have to again find the highest normalization form, and for that, we are given a relation **R(A, B, C)** with functional dependencies as follows:

{AB ->C, C ->B, AB ->B}

Candidate Key (given): **{AB}**

- Clearly, prime attributes for Relation R are: {A,B} while non-prime attributes are: {C}.
- For this particular example, let us start from the order of hierarchy with higher restrictions, and firstly, we will check for BCNF here.
- Clearly, {AB->C} and {AB->B} are in BCNF because AB is the candidate key present on the LHS of both dependencies. The second dependency, {C->B}, however, is not in BCNF because C is neither a super key nor a candidate key.
- C->B is, however, present in 3NF because B is a prime attribute that satisfies the conditions of 3NF. Hence, relation R has 3NF as the highest normalization form.

Questions:

1. Given a relation R(A, B, C, D) and Functional Dependency set FD = { AB → CD, B → C }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution :

Let us calculate the closure of AB

AB + = ABCD (from the method we studied earlier)

Since the closure of AB contains all the attributes of R, hence **AB is Candidate Key**

From the definition of Candidate Key(**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have AB as an integral part, and we have proved that AB is Candidate Key, Therefore, any superset of AB will be Super Key but not Candidate key.

Hence there will be only **one candidate key AB**

Definition of 2NF: No non-prime attribute should be partially dependent on Candidate Key

Since R has 4 attributes: - A, B, C, D, and Candidate Key is AB, Therefore, prime attributes (part of candidate key) are A and B while a non-prime attribute are C and D

a) FD: **AB** → **CD** satisfies the definition of 2NF, that non-prime attribute(C and D) are fully dependent on candidate key AB

b) FD: **B** → **C** does not satisfy the definition of 2NF, as a non-prime attribute(C) is partially dependent on candidate key AB(i.e. key should not be broken at any cost)

As FD B → C, the above table R(A, B, C, D) is not in 2NF

Convert the table R(A, B, C, D) in 2NF:

Since **FD: B → C,** our table was not in 2NF, let's decompose the table

R1(B, C)

Since the key is AB, and from FD AB → CD, we can create R2(A, B, C, D) but this will again have a problem of partial dependency B → C, hence R2(A, B, D).

Finally, the decomposed table which is in 2NF

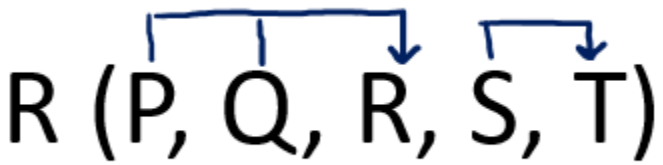
a) R1(B, C)

b) R2(A, B, D)

---

2. Given a relation R(P, Q, R, S, T) and Functional Dependency set FD = { PQ → R, S → T }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes PQS is not determined by any of the given FD, hence PQS will be the integral part of the Candidate key, i.e., no matter what will be the candidate key, and how many will be the candidate key, but all will have PQS compulsory attribute.

Let us calculate the closure of PQS

PQS + = PQSRT (from the method we studied earlier)

Since the closure of PQS contains all the attributes of R, hence **PQS is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQS as an integral part, and we have proved that PQS is Candidate Key. Therefore, any superset of PQS will be Super Key but not Candidate key.

Hence there will be only **one candidate key PQS**

Definition of 2NF: No non-prime attribute should be partially dependent on Candidate Key.

Since R has 5 attributes: - P, Q, R, S, T and Candidate Key is PQS, Therefore, prime attributes (part of candidate key) are P, Q, and S while a non-prime attribute is R and T

a) FD: **PQ → R** does not satisfy the definition of 2NF, that non-prime attribute(R) is partially dependent on part of candidate key PQS.

b) FD: **S → T** does not satisfy the definition of 2NF, as a non-prime attribute(T) is partially dependent on candidate key PQS (i.e., key should not be broken at any cost).

Hence, FD PQ → R and S → T, the above table R(P, Q, R, S, T) is not in 2NF

Convert the table R(P, Q, R, S, T) in 2NF:

Since due to FD: PQ → R and S → T, our table was not in 2NF, let's decompose the table

R1(P, Q, R) (Now in table R1 FD: PQ → R is Full F D, hence R1 is in 2NF)

R2(S, T) (Now in table R2 FD: S → T is Full F D, hence R2 is in 2NF)

And create one table for the key, since the key is PQS.

R3(P, Q, S)

Finally, the decomposed tables which is in 2NF are:

a) R1(P, Q, R)

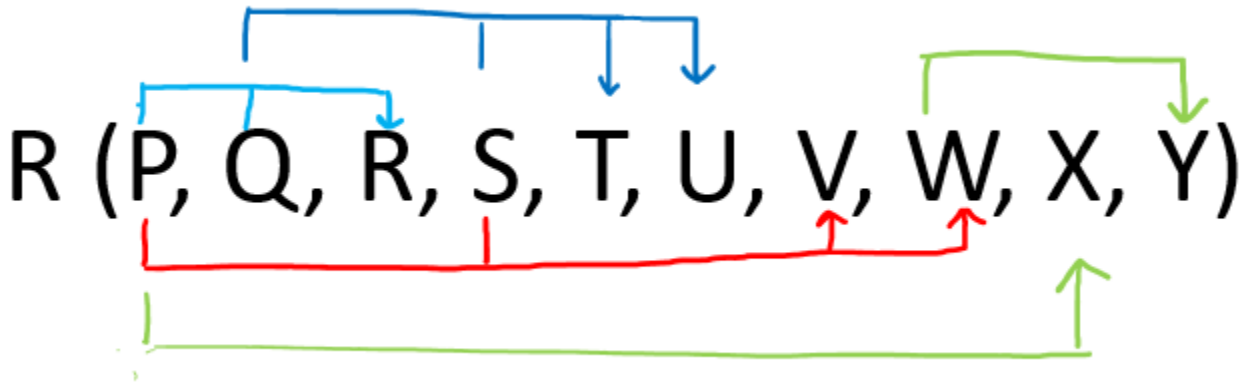
b) R2(S, T)

c) R3(P, Q, S)

---

3. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, PS → VW, QS → TU, P → X, W → Y }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes PQS is not determined by any of the given FD, hence PQS will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have PQS compulsory attribute.

Let us calculate the closure of PQS

PQS + = P Q S R T U V W X Y (from the closure method we studied earlier)

Since the closure of PQS contains all the attributes of R, hence **PQS is Candidate Key**

From the definition of Candidate Key(**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQS as an integral part, and we have proved that PQS is Candidate Key, Therefore, any superset of PQS will be Super Key but not a Candidate key.

Hence there will be only **one candidate key PQS**

Definition of 2NF: No non-prime attribute should be partially dependent on Candidate Key

Since R has 10 attributes: - P, Q, R, S, T, U, V, W, X, Y, and Candidate Key is PQS calculated using FD = { PQ → R, PS → VW, QS → TU, P → X, W → Y }. Therefore, prime attribute(part of candidate key) are P, Q, and S while non-prime attribute are R, T, U, V, W, X and Y

a. FD: **PQ → R** does not satisfy the definition of 2NF, that non-prime attribute(R) is partially dependent on part of candidate key PQS

b. FD: **PS → VW** does not satisfy the definition of 2NF, that non-prime attribute(VW) is partially dependent on part of candidate key PQS
c. FD: **QS → TU** does not satisfy the definition of 2NF, that non-prime attribute(TU) is partially dependent on part of candidate key PQS
d. FD: **P → X** does not satisfy the definition of 2NF, that non-prime attribute(X) are partially dependent on part of candidate key PQS
e. FD: **W → Y** does **not violate** the definition of 2NF, as the non-prime attribute(Y) is dependent on the non-prime attribute(W), which is not related to the definition of 2NF.

Hence because of FD: PQ → R, PS → VW, QS → TU, P → X the above table R(P, Q, R, S, T, U, V, W, X, Y) is not in 2NF

Convert the table R(P, Q, R, S, T, U, V, W, X, Y) in 2NF:

Since due to FD: PQ → R, PS → VW, QS → TU, P → X our table was not in 2NF, let's decompose the table

R1(P, Q, R) (Now in table R1 FD: PQ → R is Full F D, hence R1 is in 2NF)

R2(P, S, V, W) (Now in table R2 FD: PS → VW is Full F D, hence R2 is in 2NF)

R3(Q, S, T, U) (Now in table R3 FD: QS → TU is Full F D, hence R3 is in 2NF)

R4(P, X) (Now in table R4 FD : P → X is Full F D, hence R4 is in 2NF)

R5(W, Y) (Now in table R5 FD: W → Y is Full F D, hence R2 is in 2NF)

And create one table for the key, since the key is PQS.

R6(P, Q, S)

Finally, the decomposed tables which is in 2NF are:

R1(P, Q, R)

R2(P, S, V, W)

R3(Q, S, T, U)

R4(P, X)

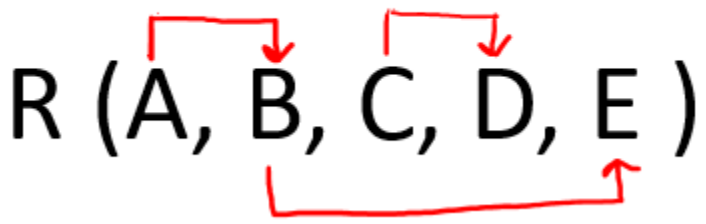
R5(W, Y)

R6(P, Q, S)

---

4. Given a relation R(A, B, C, D, E) and Functional Dependency set FD = { A → B, B → E, C → D}, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes AC is not determined by any of the given FD, hence AC will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have W compulsory attribute.

Let us calculate the closure of AC

AC + = ACBED(from the closure method we studied earlier)

Since the closure of AC contains all the attributes of R, hence **AC is Candidate Key**

From the definition of Candidate Key(**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have AC as an integral part, and we have proved that AC is Candidate Key, Therefore, any superset of AC will be Super Key but not Candidate key.

Hence there will be only **one candidate key AC**

Definition of 2NF: No non-prime attribute should be partially dependent on Candidate Key

Since R has 5 attributes: - A, B, C, D, E and Candidate Key is AC, Therefore, prime attribute (part of candidate key) are A and C while the non-prime attribute are B D and E

a. FD: **A** → **B** does not satisfy the definition of 2NF, as a non-prime attribute(B) is partially dependent on candidate key AC (i.e., key should not be broken at any cost).

b. FD: **B** → **E** does **not violate** the definition of 2NF, as a non-prime attribute(E) is dependent on the non-prime attribute(B), which is not related to the definition of 2NF.

c. FD: **C** → **D** does not satisfy the definition of 2NF, as a non-prime attribute(D) is partially dependent on candidate key AC (i.e., key should not be broken at any cost)

Hence because of FD A → B and C → D, the above table R(A, B, C, D, E) is not in 2NF

Convert the table R(A, B, C, D, E) in 2NF:

Since due to FD: A →B and C → D our table was not in 2NF, let's decompose the table

R1(A, B, E) (from FD: A → B and B → E and both are violating 2 NF definition)

R2(C, D) (Now in table R2 FD: C → D is Full F D, hence R2 is in 2NF)

And create one table for candidate key AC

R3 (A, C)

Finally, the decomposed tables which are in 2NF:

a. **R1(A, B, E)**
b. **R2(C, D)**
c. **R3(A, C)**

Procedure: To verify that given relational schema R is in 2NF or NOT, If NOT then Convert it to 2NF:

STEP 1: Calculate the Candidate Key of given R by using an arrow diagram on R.

STEP 2: Verify each FD with **Definition of 2NF** (No non-prime attribute should be partially dependent on Candidate Key)

STEP 3: Make a set of FD which do not satisfy 2NF, i.e. all those FD which are partial.

STEP 4: Convert the table R in 2NF by decomposing R such that each decomposition based on FD should satisfy the definition of 2NF:

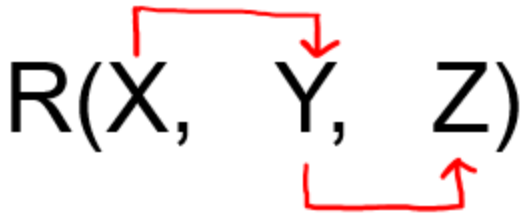
STEP 5: Once the decomposition based on FD is completed, create a separate table of attributes in the Candidate key.

STEP 6: All the decomposed R obtained from STEP 4 and STEP 5 forms the required decomposition where each decomposition is in 2NF.

---

5. Given a relation R(X, Y, Z) and Functional Dependency set FD = { X → Y and Y → Z }, determine whether the given R is in 3NF? If not convert it into 3 NF

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attribute X is not determined by any of the given FD, hence X will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have X compulsory attribute.

Let us calculate the closure of X

X + = XYZ (from the closure method we studied earlier)

Since the closure of X contains all the attributes of R, hence **X is Candidate Key**

Now check the above table is in 2 NF.

a. FD: X → Y is in 2NF (as Key is not breaking and its Fully functional dependent)
b. FD: Y → Z is also in 2NF(as it does not violate the definition of 2NF)

Hence above table R(X, Y, Z) is in 2NF but not in 3NF.

We can also prove the same from Definition 2: First, it should be in 2NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then

a. Either X is Super Key
b. Or Y is a prime attribute.

Since we have just proved that above table R is in 2 NF. Let's check it for 3NF using definition 2.

a. FD: X → Y is in 3NF (as X is a super Key)
b. FD: Y → Z is not in 3NF (as neither Y is Key nor Z is a prime attribute)

Hence because of Y → Z using definition 2 of 3NF, we can say that above table R is not in 3NF.

Convert the table R(X, Y, Z) into 3NF:

Since due to FD: Y → Z, our table was not in 3NF, let's decompose the table

FD: Y → Z was creating issue, hence one table R1(Y, Z)

Create one Table for key X, R2(X, Y), since X → Y

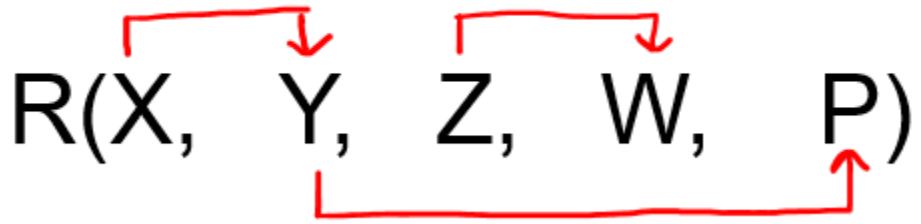
Hence decomposed tables which are in 3NF are:

R1(X, Y)

R2(Y, Z)

Question 2: Given a relation R(X, Y, Z, W, P) and Functional Dependency set FD = { X → Y, Y → P, and Z → W}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes XZ is not determined by any of the given FD, hence XZ will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have XZ compulsory attribute.

Let us calculate the closure of XZ

XZ + = XZYPW (from the closure method that we studied earlier)

Since the closure of XZ contains all the attributes of R, hence **XZ is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**).

Since all key will have XZ as an integral part, and we have proved that XZ is Candidate Key, Therefore, any superset of XZ will be Super Key but not the Candidate key.

Hence there will be only **one candidate key XZ**

Definition of 3NF: First it should be in 2NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then

a. Either X is Super Key
b. Or Y is a prime attribute.

Since R has 5 attributes: - X, Y, Z, W, P and Candidate Key is XZ, Therefore, prime attribute (part of candidate key) are X and Z while a non-prime attribute are Y, W, and P

Given FD are X → Y, Y → P, and Z → W and Super Key / Candidate Key is XZ

a. FD: **X** → **Y** does not satisfy the definition of 3NF, that neither X is Super Key nor Y is a prime attribute.
b. FD: **Y** → **P** does not satisfy the definition of 3NF, that neither Y is Super Key nor P is a prime attribute.
c. FD: **Z** → **W** satisfies the definition of 3NF, that neither Z is Super Key nor W is a prime attribute.

<u>Convert the table R(X, Y, Z, W, P) into 3NF:</u>

Since all the FD = { X → Y, Y → P, and Z → W} were not in 3NF, let us convert R in 3NF

R1(X, Y) {Using FD X → Y}

R2(Y, P) {Using FD Y → P}

R3(Z, W) {Using FD Z → W}

And create one table for Candidate Key XZ

R4(X, Z) { Using Candidate Key XZ }

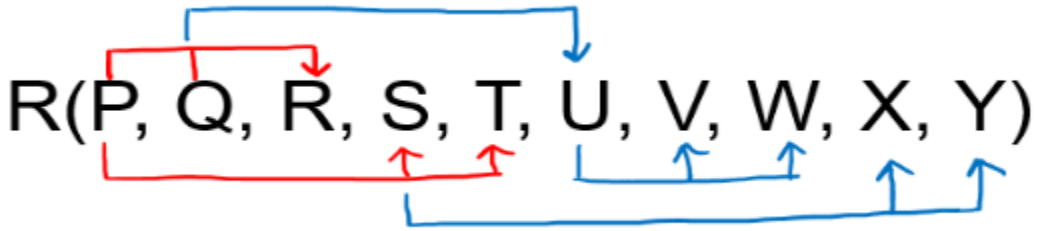
All the decomposed tables R1, R2, R3, and R4 are in 2NF(as there is no partial dependency) as well as in 3NF.

Hence decomposed tables are:

R1(X, Y), R2(Y, P), R3(Z, W), and **R4(X, Z)**

Question 3: Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, P → ST, Q → U, U → VW, and S → XY}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attribute PQ is not determined by any of the given FD, hence PQ will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have PQ compulsory attribute.

Let us calculate the closure of PQ

PQ + = P Q R S T U X Y V W (from the closure method we studied earlier)

Since the closure of XZ contains all the attributes of R, hence **PQ is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQ as an integral part, and we have proved that XZ is Candidate Key, Therefore, any superset of PQ will be Super Key but not Candidate key.

Hence there will be only **one candidate key PQ**

Definition of 3NF: First it should be in 2NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then

c) Either X is Super Key

d) Or Y is a prime attribute.

Since R has 10 attributes: - P, Q, R, S, T, U, V, W, X, Y, V, W and Candidate Key is PQ, Therefore, prime attribute (part of candidate key) are P and Q while a non-prime attribute are R S T U V W X Y V W

Given FD are {PQ → R, P → ST, Q → U, U → VW and S → XY} and Super Key / Candidate Key is PQ

a. FD: **PQ → R** satisfy the definition of 3NF, as PQ Super Key
b. FD: **P → ST** does not satisfy the definition of 3NF, that neither P is Super Key nor ST is the prime attribute
c. FD: **Q → U** does not satisfy the definition of 3NF, that neither Q is Super Key nor U is a prime attribute
d. FD: **U → VW** does not satisfy the definition of 3NF, that neither U is Super Key nor VW is a prime attribute
e. FD: **S → XY** does not satisfy the definition of 3NF, that neither S is Super Key nor XY is a prime attribute

Convert the table R(X, Y, Z, W, P) into 3NF:

Since all the FD = { P → ST, Q → U, U → VW, and S → XY } were not in 3NF, let us convert R in 3NF

R1(P, S, T) {Using FD P → ST }

R2(Q, U) {Using FD Q → U }

R3(U, V, W) { Using FD U → VW }

R4(S, X, Y) { Using FD S → XY }

R5(P, Q, R) { Using FD PQ → R, and candidate key PQ }

All the decomposed tables R1, R2, R3, R4, and R5 are in 2NF(as there is no partial dependency) as well as in 3NF.

Hence decomposed tables are:

R1(P, S, T), R2(Q, U), R3(U, V, W), R4(S, X, Y), and **R5(P, Q, R)**